KEY run sheet ·flow A: paper dry run

Cut one practice key from start to finish using numbers and dots only, punch its lock, then prove both by restoring the key in a wallet. Two people: one **cutter**, one **checker**. Allow about an hour and a half.

> **Practice only. This key is made on paper and typed into a computer. It must never hold money, and everything is destroyed at the end.**

On the table

- Eleven dice (2 orange, 5 white, 4 black) and the dice strips
- Number sheets A to D
- One paper seed plate (seed plate design, page 2) and a dark pen
- This sheet

Kept aside

- **From Part C:** a computer with the LOCK repository, which holds `KEY/lock_standin.py`
- **From Part E:** one paper lock plate (lock plate design, page 2) and the address card
- **From Part F:** word sheets A to D, a paper slip, and a phone with a wallet app

Before you start, test the stand-in with the worked example. Run it with the eleven numbers on the seed plate example (seed plate design, page 3) and the bits 1010010. It must answer row 12 = 1313, and its nineteen lock rows must equal the lock plate example (lock plate design, page 3).

The three rules

- **Dots on top = ● = mark the cell. Blank on top = ○ = leave the cell empty.** (Ordinary dice: even ●, odd ○.)
- **Each die has one place:** its colour gives the group, its dot count gives the position, fewest dots on the left. Nobody chooses where a die goes.
- **Each die is thrown once per row.** No second tries.

Part A ·Rows 1 to 11 (do these five steps eleven times)

- ☐ **1** **Throw** all eleven dice once.
- ☐ **2** **Place** each die in its own slot on the strip, without turning it.
- ☐ **3** **Mark the row.** Cutter: for each die showing dots, fill the matching cell in this row of the plate. Leave the others empty.
- ☐ **4** **Find the number.** Orange dice give the sheet (A to D), white dice the row, black dice the column. Write the four digits in the box at the end of the row.
- ☐ **5** **Check.** Checker: compare the plate row with the dice, cell by cell, then confirm the number on the sheet. Tick the log. Only then clear the dice.

Part B ·Row 12, first seven cells

- ☐ **6** **Throw** the two orange and five white dice only. Place them on the row 12 strip.
- ☐ **7** **Mark** the first seven cells of row 12. Checker compares them with the dice.
- ☐ **8** **Leave** the four shaded cells and the number box of row 12 empty.

Log

Row	1	2	3	4	5	6	7	8	9	10	11	12
Dots match dice												
Number written												
Final check (Part D)												

Start: _____ End of B: _____ End of D: _____ End of E: _____ End of F: _____

Part C ·The checksum, from the LOCK stand-in

On the real machine this part is switches and lights. For the dry run a computer does the same sum. It takes the plate's numbers and gives back dots and numbers. No words are used.

- [ ] **9** **Type** the eleven numbers from the plate, then the seven cells of row 12 as 1 for a dot and 0 for a blank, with no spaces. In the LOCK folder:
  `python3 KEY/lock_standin.py n1 n2 … n11 bbbbbbb`
- [ ] **10** **Check what it read.** It lists the eleven rows back as dots and numbers. Checker compares them with the plate.
- [ ] **11** **Read "Row 12".** Its first seven dots must equal the seven cells already marked. If they differ, stop and find the mistake. Then **mark the last four** in the shaded cells.
- [ ] **12** **Find row 12's number** on the number sheets from its eleven cells, write it in the box, and only then compare it with the number on the screen.

Part D ·Whole-plate check

- [ ] **13** **Checker alone**, with the seed plate and the number sheets only. For each of the twelve rows: read the dots, find that sheet, row and column, and compare the number printed there with the number in the box.
- [ ] **14** **Tick** the last line of the log. All twelve must agree before going on. On steel, this is the last moment to find a mistake before sealing.

Part E ·The lock plate

The stand-in has also printed THE LOCK: nineteen rows, as LOCK's lights would show them. They are the key's one address, and nothing in them is secret.

- [ ] **15** **Copy.** One person reads each row aloud, light by light; the other fills a cell for each `#`. After each row, count the dots: the number must be even.
- [ ] **16** **Number.** In each group of five, add the weights above the dots (16, 8, 4, 2, 1) and write the two numbers in the boxes. Only then compare them with the numbers on the screen.
- [ ] **17** **Write the address.** Write bc1q, then look up each number on the address card, row by row. Compare the finished address with the one on the screen.

Part F ·Sweep test

This proves the whole chain: that a seed plate holding only dots and numbers opens the address on its lock plate. It is the first time words appear.

- ☐ **18** **Look up the words.** A different person takes the seed plate and the word sheets, finds all twelve numbers, and writes twelve words on the slip.
- ☐ **19** **Restore** the twelve words in the wallet app as an imported wallet. If asked for a type, choose native SegWit (BIP84).
- ☐ **20** **Compare.** Ask the wallet for its receiving address. On a wallet that has never received anything, this is its first address, and it must equal the address written from the lock plate.
- ☐ **21** **Result:** ☐ match ☐ no match. If there is no match, check the word lookup first, then the seed plate rows against the screen.

Flow B, the SeedSigner check, comes here. If you are doing it, stop now and follow its run sheet before destroying anything.

Part G ·Destroy and record

- ☐ **22** **Delete** the wallet from the app. Close the program and clear the screen.
- ☐ **23** **Destroy** the seed plate and the word slip. Destroy the lock plate too: it holds nothing secret, but nobody should ever pay to a practice address.
- ☐ **24** **Record** the date, who took part, the result and the times. Never the numbers or the words.

Notes for the next version

Which step was slowest? Where did a mistake happen, and which check caught it? Was any instruction unclear?